arXiv:2607.28483v1 [cs.CV] 30 Jul 2026

Towards Real-Time PixOOD: Efficient Anomaly Segmentation for Autonomous Vehicles

Luca de Martino, Federico Aromolo, Federico Nesti, and Giorgio Buttazzo

Department of Excellence in Robotics & AI, Scuola Superiore Sant'Anna, Pisa, Italy

Abstract. Real-time anomaly segmentation is essential for the safety of autonomous systems. Although recent approaches offer high accuracy, their computational cost limits their deployment on embedded hardware. This work presents an efficient and accelerated pipeline designed for both embedded and desktop platforms, targeting the autonomous driving and railway domains. The proposed approach reformulates the Neyman–Pearson scoring stage of PixOOD, a state-of-the-art out-of-distribution detection method, and deploys the full pipeline through hardware-optimized TensorRT compilation, reaching up to 182 FPS on a desktop NVIDIA RTX 4060 GPU and 75 FPS on the NVIDIA Jetson AGX Orin embedded platform, respectively 20× and 18× faster than the original baseline. The achieved results demonstrate that advanced anomaly segmentation can be efficiently deployed for onboard processing in autonomous driving and railway applications.

Keywords: Real-time Anomaly Segmentation · Embedded Systems · Railway Safety · Autonomous Driving · Hardware Acceleration

1 Introduction

In autonomous driving and railway applications, the ability to identify previously unseen objects at the pixel level, such as obstacles on the tracks or unexpected debris on the roadway, is essential for safe operation [14,12]. Anomaly segmentation addresses this challenge by producing a dense per-pixel score map that flags pixels deviating from the learned distribution, thus enabling downstream safety modules to trigger appropriate responses. Performing such segmentation in real time is therefore crucial, as autonomous driving systems must respond faster than a human driver [7]; as a result, this work targets 25 FPS.

The research community has primarily focused on providing anomaly segmentation methods based on different sensor inputs [15,3] and on establishing benchmarks that measure anomaly segmentation performance metrics, such as Average Precision (AP) and false positive rate at 95 % recall (FPR95). However, the characterization of real-time deployment on embedded hardware remains largely unexplored. The railway domain has received even less attention in this regard and, to the best of our knowledge, OSDaR-AR [12] represents the only publicly available benchmark for autonomous railway operations that provides

pixel-level anomaly annotations, along with LiDAR point clouds calibrated and synchronized with RGB frames.

Among the families of anomaly segmentation methods, prototype-based approaches model the in-distribution manifold and flag statistical deviations as anomalous, without requiring explicit outlier exposure during training [15]. A representative method of this family is PixOOD [18], ranked first, at the time of writing, in the LostAndFound (LaF) [14] benchmark for autonomous driving [15]. However, the original PixOOD implementation relies on DINOv2 [13], a Vision Transformer (ViT) backbone, in its largest version, ViT-L. Since DINOv2 is available in several model sizes, this work adopts the smaller ViT-S version, which is better suited to embedded systems, and applies the same choice to the more recent DINOv3 [16]. Even with this lighter backbone and a reduced input resolution of 640 px, the embedded target reaches only 4.26 FPS, far below the 25 FPS requirement, because the CPU scoring stage dominates the per-frame latency. Specifically, the Neyman–Pearson scoring stage runs entirely on the CPU in the original implementation, preventing real-time operation on the embedded target. These findings highlight two compounding limitations: (i) state-of-the-art anomaly segmentation methods do not leverage the hardware acceleration technologies necessary to meet real-time requirements, and (ii) standard benchmarks only evaluate detection accuracy without reporting deployment-relevant latency or energy consumption.

Paper contributions. To overcome these limitations, this paper provides three main contributions targeting the efficient deployment of PixOOD:

- A GPU reimplementation of the Neyman–Pearson scoring stage that evaluates the per-class density in log-space and replaces the CPU cumulative-distribution lookup with a GPU-native operator, eliminating the per-frame CPU round-trip; reducing the per-frame latency at 640 px by 3.78× on the NVIDIA Jetson AGX Orin embedded GPU platform and by 4.66× on the NVIDIA RTX 4060 desktop GPU, with less than 0.15 % AP deviation from the original implementation.
- A hardware-aware deployment pipeline using TensorRT (a high-performance inference framework by NVIDIA) compilation via an ONNX-to-TensorRT workflow (ONNX is an interoperable format for neural network models) on both the Jetson AGX Orin and the desktop NVIDIA RTX 4060 GPU, with FP16 and FP32 precision selected per model architecture, reaching up to 182 FPS on the desktop GPU and 75 FPS on the embedded platform, up to 20× over the original baseline.
- A systematic cross-domain characterization of detection accuracy, latency, and energy on an embedded and a desktop platform, across the autonomous driving and railway domains.

Paper organization. The remainder of this paper is organized as follows: Section 2 reviews the relevant related work; Section 3 details the background and system model; Section 4 describes the proposed optimizations; Section 5 reports the experimental evaluation; and Section 6 presents the conclusions and future directions.

2 Related work

Building on the categorization of Shoeb et al. [15], this work groups anomaly segmentation approaches into the following five families.

Uncertainty-based methods. These approaches derive a scalar anomaly score at test time from the output of a semantic segmentation network, without modifying its architecture. Representative scoring functions include the standardized max logit [6] and the energy score [10]. To improve discriminability, several state-of-the-art methods in this family additionally fine-tune the network with auxiliary outlier exposure, typically the COCO dataset [8,15].

Mask-based methods. These methods rely on a mask-transformer decoder that produces a set of mask queries, each associated with a binary mask and a class distribution. A pixel is flagged as anomalous when no query of a known class claims it [11]. The computational cost of the query-based decoder limits the deployment of these methods on real-time embedded systems [15].

Reconstructive methods. These methods learn to reconstruct the input from a closed-set prediction and compute the anomaly score from the discrepancy between the original input and its reconstruction [9]. They do not require outlier exposure but are prone to false positives in complex or repetitive textures [15].

Foundation model approaches. These methods adapt large-scale vision foundation models to segment out-of-distribution objects [20]. However, the high computational cost of running such large models on embedded hardware prevents their real-time deployment [15].

Prototype and density-based methods. These methods parameterize in-distribution class manifolds and detect anomalies as statistical deviations from these models [18]. Since these approaches rely exclusively on a model of in-distribution normality and require no anomalous data, they can be applied to new domains by retraining on in-distribution data alone.

Among the methods described above, PixOOD [18], a prototype-based method, achieves state-of-the-art accuracy on autonomous driving benchmarks without outlier exposure, yet its deployment characteristics on embedded hardware have not been systematically characterized. More generally, to the best of our knowledge, no published work provides a systematic latency and energy characterization of anomaly segmentation methods on embedded automotive or railway hardware. This work fills this gap by reformulating PixOOD into a deployment-ready pipeline and benchmarking it on representative embedded and desktop platforms, across both the autonomous driving and railway domains.

3 Background

This section describes the datasets and the evaluation protocols adopted in this work, also providing a brief overview of the PixOOD architecture, used as the baseline approach.



Fig. 1. Two examples of anomaly segmentation. Red overlays mark the pixels flagged as anomalous. Left: an OSDaR-AR railway scene, where a cow placed on the tracks is correctly detected as an out-of-distribution obstacle. Right: a LostAndFound road scene, where a small obstacle on the road surface is flagged as an anomaly.

3.1 Datasets and evaluation protocols

Standard anomaly segmentation benchmarks for autonomous driving, such as LostAndFound [14], Fishyscapes [2], and SegmentMeIfYouCan (SMIYC) [4], adopt AP (↑ higher is better) and FPR95 (↓ lower is better) as primary evaluation metrics. These metrics are computed only within a road-region mask: pixels outside the road and sidewalk (e.g., sky, buildings, and vegetation) receive an *ignore* label and are excluded from all calculations. While this facilitates comparisons among autonomous driving methods, it conceals detections that would trigger spurious alarms when the full image is processed.

Among the available autonomous driving benchmarks, LaF is selected as the primary evaluation set for this work because, unlike Fishyscapes and SMIYC, it provides complete pixel-level annotations for all objects in the test set, which allows evaluating the full frame rather than only the road-region mask.

LostAndFound contains 1203 evaluation frames at 2048×1024 pixels showing small road obstacles. Because LaF provides complete pixel-level annotations, the ignore labels can be removed to define a full-frame protocol that reflects deployment conditions, in which the entire image is processed. In this work, LaF is therefore evaluated under two protocols: the *standard* road-region protocol, for comparability with published rankings, and the *full-frame* protocol, which reveals the off-road behavior of the model (Section 5).

OSDaR-AR [12] augments real railway scenes with synthetic out-of-distribution obstacles through augmented reality, providing 1800 frames at 2464×1600 px. The obstacles, overlaid as the only annotated objects, make the ground truth binary and full-frame, with no ignore labels. OSDaR-AR is therefore evaluated under the full-frame protocol only. Figure 1 shows qualitative examples of the accelerated PixOOD pipeline on both datasets.

3.2 PixOOD architecture

The PixOOD processing pipeline is summarized in Figure 2 and consists of a frozen ViT backbone, an MLP decoder and a K-Nearest Neighbors (KNN) prototype

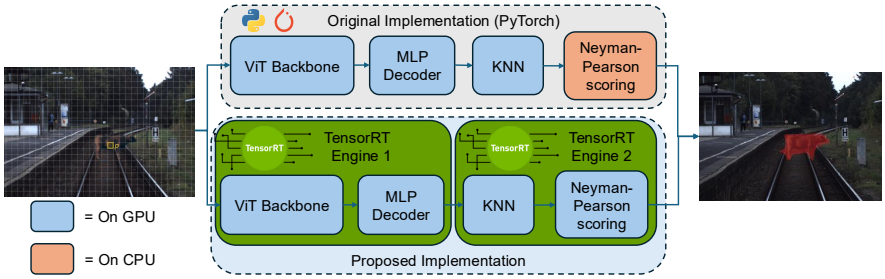


Fig. 2. PixOOD processing pipeline. *Top*: the original PyTorch implementation [18], in which the ViT backbone, MLP decoder, and KNN prototype module run on the GPU (blue), while the Neyman–Pearson scoring stage runs on the CPU (orange). *Bottom*: the proposed accelerated implementation, organized into two TensorRT engines that run entirely on the GPU.

module, both applied to each patch embedding z to produce its two-component vector z_c, and a Neyman–Pearson scoring stage based on the likelihood ratio. Each component is described in the following paragraphs.

ViT backbone. PixOOD uses DINOv2 as a feature extractor backbone, whose weights remain fixed after pre-training. The input image is divided into non-overlapping patches whose size depends on the specific backbone variant (14×14 px for DINOv2 and 16×16 px for the more recent DINOv3), and each patch is mapped to a D-dimensional embedding vector z.

MLP decoder. A four-layer Multi-Layer Perceptron (MLP) projects each patch embedding z into a class-specific logit $f_c(z)$ for each class c of the C semantic classes of the training distribution.

Prototype condensation and KNN. During training, a differentiable soft k-means algorithm clusters the patch embeddings into $K = 1000$ prototypes per semantic class, providing a compact representation of the feature space within each class distribution. At inference, a batched KNN module computes the L2 distance $d(z, k_c^*)$ from each patch embedding z to its nearest prototype k_c^* of class c. Together with the per-class MLP logit $f_c(z)$, this distance forms a 2D vector z_c:

$$z_c = \big(f_c(z),\, d(z, k_c^*)\big). \tag{1}$$

Neyman–Pearson score. For each semantic class c, a two-dimensional Gaussian distribution $\mathcal{N}(\mu_c, \Sigma_c)$ is fitted on the coordinate vectors z_c of in-distribution patches. The anomaly score follows the generalized Neyman–Pearson (N-P) likelihood ratio $r(z_c)$ between the fitted in-distribution density and a reference out-of-distribution density, the optimal statistic for separating in-distribution from out-of-distribution patches [18]. To obtain a calibrated score, the ratio is mapped through its cumulative distribution function (CDF) over the in-distribution data of class c, denoted F_c. This function returns the fraction of in-distribution patches

whose ratio does not exceed $r(z_c)$, a calibrated value in $[0, 1]$ that is high for patches resembling class c and low otherwise. F_c has no closed form and is therefore precomputed as a lookup table, which at inference is evaluated by interpolation at the ratio of each patch. The final per-pixel score $s(x)$ at image location x is:

$$s(x) = 1 - \max_c F_c\big(r(z_c(x))\big). \tag{2}$$

Intuitively, a patch is flagged as anomalous when its coordinate z_c lies in a low-probability region of every in-distribution class density, i.e., when no known class can plausibly explain it.

4 Proposed optimizations

This section presents two optimizations that transform PixOOD into a deployment-ready pipeline: a GPU-accelerated N-P scoring reformulation (Section 4.1) and a TensorRT-based deployment (Section 4.2).

Figure 2 contrasts the proposed TensorRT-based implementation with the original PyTorch-based baseline.

The original PixOOD implementation exhibits a severe CPU bottleneck in the Neyman–Pearson scoring stage. The CDF lookup is performed on the CPU with SciPy [18] in double precision floating point (float64), requiring a round-trip data transfer per frame: the patch embeddings computed on the GPU are transferred to host memory, where SciPy evaluates the multivariate normal density and performs 1D interpolation on the precomputed CDF tables sequentially for each of the C classes. The resulting anomaly scores are then transferred back to the GPU. On the Jetson AGX Orin at 640 px input resolution, this round-trip alone dominates the 234.7 ms per-frame inference time, yielding 4.26 FPS. The GPU reimplementation described in Section 4.1 eliminates this round-trip by running the scoring stage on the GPU.

4.1 GPU-accelerated Neyman–Pearson scoring

The original scoring implementation exhibits three crucial bottlenecks: (i) the CPU round-trip described above, (ii) a multivariate normal probability density computation performed in float64 and serialized over the C semantic classes (i.e., one class evaluated at a time, with no parallel exploitation across classes), and (iii) a CPU-bound CDF interpolation routine that is neither parallelizable nor exportable to ONNX. The proposed GPU reimplementation addresses all three bottlenecks through two complementary components: (i) a log-space GPU kernel for the multivariate normal density, and (ii) a GPU-native CDF lookup.

Log-space multivariate normal distribution. To exploit the GPU efficiently, the probability density computation uses single-precision floating point (float32), the native precision of GPU compute cores, rather than the double-precision float64 of the original CPU implementation, since float64 throughput is drastically

lower on consumer and embedded GPU hardware. In the original CPU implementation, the density is evaluated in linear space using float64, which avoids underflow thanks to the wider representable range compared to float32. The linear-space density of the per-class 2D Gaussian $\mathcal{N}(\mu_c, \Sigma_c)$ might underflow the float32 representable range when the covariance determinant $|\Sigma_c|$ is small. The proposed GPU kernel therefore reformulates the computation entirely in log-space: $\log p(z_c \mid c) = -\frac{1}{2}\big(d_M^2 + 2\log(2\pi) + \log|\Sigma_c|\big)$, where $d_M^2 = (z_c - \mu_c)^\top \Sigma_c^{-1}(z_c - \mu_c)$ is the squared Mahalanobis distance. Since Σ_c is fixed after training and is only 2×2, both Σ_c^{-1} and $\log|\Sigma_c|$ are precomputed offline. The resulting tensors are stored on the GPU, and the log-probability for all C classes is then evaluated in a single batched operation across all patches, with no CPU involvement.

GPU-native CDF lookup. The original implementation evaluates the one-dimensional CDF lookup table on the CPU. For each class, it first computes the per-patch likelihood ratios from their coordinates z_c and then maps each ratio to a score by piecewise-linear interpolation (`scipy.interpolate.interp1d`), processing one class at a time. Two GPU variants are considered to replace it. The first variant uses `torch.searchsorted`, a GPU binary-search operator that replicates the original piecewise-linear CDF interpolation in a single batched kernel over all patches and classes. This approach results in AP and FPR95 values that closely match the ones reported in the original paper. Unfortunately, `torch.searchsorted` is not part of the ONNX operator set and therefore cannot be compiled into a TensorRT engine in the proposed workflow, so it is retained only as an accuracy reference. The second variant is precomputed offline for deployment, relying on `grid_sample`, an operator in the ONNX operator set allowing direct compilation into a TensorRT engine. For each class c, the complete scoring sequence of Section 3, from the evaluation of $p(z_c \mid c)$ to the calibrated score $F_c(r(z_c))$, is evaluated over a uniform 200×200 grid of z_c coordinates, and the resulting score maps, one per class, are stored on the GPU as static weights. At inference, the scoring sequence is not re-evaluated. Since the maps are defined on a discrete grid while z_c takes continuous values, the score is obtained by bilinear interpolation among the four surrounding grid nodes via `grid_sample` on the GPU. The choice of 200×200 nodes per class balances accuracy and memory, keeping the deviation from the reference negligible while the score maps fit entirely in the GPU cache.

Latency and accuracy. Running this scoring stage entirely on the GPU removes the per-frame CPU round-trip. Using a PyTorch implementation at 640 px input resolution, this lowers the per-frame latency from 234.7 ms to 62.1 ms on the Jetson AGX Orin (4.26 FPS to 16.1 FPS, a 3.78× speedup) and from 106.0 ms to 22.8 ms on the desktop RTX 4060 (9.4 FPS to 44.0 FPS, a 4.66× speedup), with less than 0.15 % AP deviation from the original implementation. Further latency improvements are provided by leveraging TensorRT.

4.2 Hardware-optimized deployment

The GPU N-P reformulation of Section 4.1 achieves a throughput of 16.1 FPS on the Jetson AGX Orin, which is still below the 25 FPS target. Closing this

gap requires compiling the full pipeline into TensorRT engines [1], rather than leveraging PyTorch for GPU inference. The inference graph is split into two engines executed in sequence: the first contains the ViT backbone and the MLP decoder, the second the KNN module and the `grid_sample` variant of the Neyman–Pearson scoring stage.

ONNX export and attention fix. Both engines are produced from ONNX exports of their source PyTorch modules: the backbone with the MLP decoder forms one graph, and the KNN with the `grid_sample` N-P scoring stage forms the other. The backbone export requires an additional fix to ensure attention stability under TensorRT compilation. DINOv2 and DINOv3 implementations use fused scaled dot-product attention (SDPA). When this fused kernel is compiled into TensorRT and executed in FP16 on the Jetson AGX Orin, it becomes numerically unstable and collapses the attention output to a near-zero cosine similarity with respect to the PyTorch reference, a failure mode also reported by Turkcan [17]. This instability does not arise on the desktop RTX 4060 GPU. To address it, the proposed implementation replaces the fused SDPA with an explicit, non-fused implementation before export, preventing TensorRT from selecting the unstable fused kernel. The fix is nonetheless applied to all model architectures on both platforms to keep a single implementation in the evaluation.

Desktop deployment (RTX 4060). The standard ONNX-to-TensorRT workflow is used. DINOv2 is compiled in half-precision floating-point (FP16) without numerical issues. In contrast, DINOv3 relies on rotary positional embeddings, which PyTorch exports as conditional `If` nodes. Since the TensorRT compilation of these nodes is numerically unstable in FP16, DINOv3 is deployed in FP32.

Embedded deployment (Jetson AGX Orin). On the Orin, the ONNX Runtime distribution available for this platform is linked against cuDNN 8, used for accelerating neural operations on GPU, whereas JetPack 6 (the NVIDIA system software stack for Jetson platforms) ships with cuDNN 9. Since the two versions are not binary-compatible, the TensorRT engines are compiled from ONNX and executed through the native TensorRT Python API instead. On both platforms, the deployed scoring stage uses the `grid_sample` variant of Section 4.1, the only one of the two GPU variants that can be exported to ONNX.

5 Experimental results

This section reports empirical results covering detection accuracy under different evaluation protocols on the autonomous driving domain (LostAndFound) and the railway domain (OSDaR-AR), real-time performance on two hardware platforms, and energy consumption on both platforms.

5.1 Experimental setup

The system is evaluated with four trained model variants, all using the ViT-S backbone: DINOv2 and DINOv3, each trained on Cityscapes [5] (19 urban semantic classes) and RailSem19 [19] (19 railway semantic classes), so $C = 19$ for

Table 1. LostAndFound detection accuracy at native resolution (2048 × 1024 px). The full-frame protocol removes all ignore labels. All models are trained on Cityscapes. The best value of each metric within each protocol is shown in bold.

Model	**Road Region**		**Full Frame**	
	AP ↑	FPR95 ↓	AP ↑	FPR95 ↓
DINOv2-CS	**87.46**	**1.81**	0.20	70.49
DINOv3-CS	78.86	14.61	**0.31**	**62.11**

both. These variants are denoted DINOv2-CS, DINOv2-RS, DINOv3-CS, and DINOv3-RS, where CS and RS indicate the Cityscapes and RailSem19 training sets, respectively. Throughout, the resolution label denotes the largest image dimension after resizing. Each side is then rounded down to the nearest multiple of the backbone patch size, as required by the ViT, and the exact input dimensions therefore depend on the backbone and on the image aspect ratio. LostAndFound is evaluated at its native resolution (2048 × 1024 px) under both the standard road-region protocol and the full-frame protocol described in Section 3.1. OSDaR-AR is evaluated exclusively under the full-frame protocol at three resolutions: 1792 px (highest evaluated resolution), 896 px (intermediate accuracy-throughput trade-off), and 640 px (embedded deployment target, primary configuration on the Jetson AGX Orin).

Hardware validation is conducted on two platforms: (i) a desktop workstation equipped with an NVIDIA RTX 4060 GPU (8 GB GDDR6), an Intel Core i7-14700F CPU, and 32 GB of DDR5 system RAM; and (ii) an NVIDIA Jetson AGX Orin (64 GB unified LPDDR5, 12-core ARM Cortex-A78AE CPU) operating in the MAXN power mode, which enables full SoC performance. Per-frame inference latency is measured using CUDA synchronization events, averaged over 2000 inference iterations. Image loading and normalization are not included in the reported time.

5.2 Evaluation protocol and detection accuracy

Table 1 presents detection accuracy on LostAndFound under both evaluation protocols. Under the standard road-region protocol, DINOv2-CS achieves 87.46 % AP, consistent with the published PixOOD baseline that uses the larger ViT-L backbone [15]. This confirms that the lighter ViT-S backbone adopted in this work preserves detection accuracy, and it is therefore used in all subsequent experiments. DINOv3-CS achieves lower road-region accuracy (78.86 % AP vs. 87.46 % AP), likely because its larger 16×16 px patches reduce spatial granularity at the same input resolution.

Under the full-frame protocol, the AP of both variants collapses below 1 %. This result stems entirely from the off-road regions that the road-region protocol excludes but that a deployed system must process, where the models produce numerous false positives. The standard protocol is therefore optimistic with respect to real deployment, motivating the use of genuinely full-frame benchmarks such as OSDaR-AR.

Table 2. Detection accuracy on OSDaR-AR at three evaluation resolutions. All models are trained on RailSem19. The best value of each metric at each resolution is shown in bold.

Model	1792 px		896 px		640 px	
	AP ↑	FPR95 ↓	AP ↑	FPR95 ↓	AP ↑	FPR95 ↓
DINOv2-RS	43.62	30.07	**46.77**	**31.63**	**39.51**	**37.43**
DINOv3-RS	**56.75**	**29.77**	24.88	45.63	10.60	56.42

Table 3. Original baseline vs. TensorRT pipeline on the desktop NVIDIA RTX 4060 (*4060*) and the NVIDIA Jetson AGX Orin (*Orin*, MAXN power mode), averaged over 2000 inference instances. *Baseline* columns are the original SciPy CPU scoring stage; *TensorRT* columns are the deployed pipeline, at the same precision. The baseline energy does not include the CPU scoring cost and is therefore a lower bound. For each platform, the best value in each column is shown in bold.

Platf.	Model	Res.	Prec.	Speed (ms / FPS)		Energy (J)	
				Baseline	TensorRT	Baseline	TensorRT
4060	DINOv2	640 px	FP16	**109.4 / 9.1**	**5.51 / 181.6**	**6.23**	**0.56**
	DINOv2	896 px	FP16	205.7 / 4.9	12.44 / 80.4	11.83	1.36
	DINOv2	640 px	FP32	115.5 / 8.7	11.93 / 83.8	7.15	1.31
	DINOv2	896 px	FP32	232.4 / 4.3	37.97 / 26.3	14.88	4.30
	DINOv3	640 px	FP32	119.3 / 8.4	9.38 / 106.6	7.16	1.02
	DINOv3	896 px	FP32	225.6 / 4.4	26.10 / 38.3	13.82	2.93
Orin	DINOv2	640 px	FP16	**240.2 / 4.2**	**13.34 / 75.1**	**1.80**	**0.44**
	DINOv2	896 px	FP16	447.6 / 2.2	28.16 / 35.5	3.37	1.00
	DINOv2	640 px	FP32	260.4 / 3.8	25.60 / 39.1	2.01	0.89
	DINOv2	896 px	FP32	504.4 / 2.0	70.98 / 14.1	4.13	2.63
	DINOv3	640 px	FP32	306.4 / 3.3	21.73 / 46.0	2.52	0.71
	DINOv3	896 px	FP32	555.0 / 1.8	50.85 / 19.7	4.65	1.83

Table 2 reports OSDaR-AR accuracy across three resolutions. DINOv3-RS achieves the best accuracy at 1792 px (56.75 % AP, 29.77 % FPR95), while DINOv2-RS peaks at 896 px (46.77 % AP, 31.63 % FPR95), slightly outperforming its 1792 px result (43.62 % AP) due to increased patch density in the anomaly region at the intermediate resolution. DINOv3-RS degrades significantly below 1792 px since its larger patch size generates fewer patches at lower resolutions, reducing the spatial granularity available to detect the small anomaly stickers.

5.3 Real-time performance and energy efficiency

The full proposed pipeline is compiled with TensorRT, as described in Section 4.2. Table 3 reports the per-frame latency, throughput, and energy on both platforms. For comparison, Table 3 also reports the latency and energy of the original baseline, over which the deployed pipeline is 6–20× faster and consumes less energy per frame at matching precision. On the Jetson AGX Orin, the DINOv2-RS FP16 pipeline reaches 75.1 FPS at the 640 px label and 35.5 FPS at 896 px, comfortably exceeding the 25 FPS safety target, while the DINOv3-RS FP32

pipeline reaches 46.0 FPS at 640 px. On the RTX 4060, the same two configurations reach 181.6 FPS and 106.6 FPS at 640 px, respectively. As discussed in Section 4.2, DINOv3 runs in FP32 on both platforms because of the FP16 instability of its rotary positional embeddings. Table 3 also reports DINOv2-RS in FP32 as a precision reference. At 640 px and equal precision, DINOv3-RS is faster than DINOv2-RS on both platforms (21.73 ms vs. 25.60 ms on the Orin, 9.38 ms vs. 11.93 ms on the RTX 4060), but with lower detection accuracy reported in Table 2. The FP16 backbone provides a 1.92× end-to-end speedup over its FP32 counterpart on the Orin at 640 px (13.34 ms vs. 25.60 ms). On the Jetson AGX Orin, energy is characterized through the on-chip `tegrastats` monitor, which samples the SoC GPU power rail (`VDD_GPU_SOC`) at 100 ms intervals. Energy per frame is then obtained by multiplying the mean power draw by the per-frame inference time. The DINOv2-RS FP16 configuration consumes 0.44 J per frame at the 640 px label (32.94 W mean draw), rising to 1.00 J at 896 px. On the RTX 4060, the available sensors report the total board power, which includes the GPU die, the GDDR6 memory, and the on-board voltage regulators. The corresponding values (0.56 and 1.36 J for DINOv2-RS FP16) are therefore board-level. The Orin energy is measured at the chip-level GPU power rail, whereas the RTX 4060 energy reflects the total board power and is therefore not directly comparable.

6 Conclusions

This work presented a systematic benchmark of PixOOD on both desktop and embedded platforms, demonstrating its viability for real-time deployment across two domains, autonomous driving and railway operations. The GPU reimplementation of the Neyman–Pearson scoring stage achieved a 3.78× end-to-end speedup on the Jetson AGX Orin, from 4.26 FPS to 16.1 FPS. Combined with platform-specific TensorRT compilation, the full pipeline reaches up to 182 FPS on the RTX 4060 and 75 FPS on the Orin (DINOv2-RS FP16 at the 640 px label, 0.44 J per frame), well above the 25 FPS real-time target on both platforms. The dual-protocol evaluation of LostAndFound further showed that the standard road-region protocol is optimistic with respect to real deployment conditions, motivating the adoption of full-frame benchmarks such as OSDaR-AR.

To the best of our knowledge, the existing literature on anomaly segmentation focuses primarily on detection accuracy for autonomous driving, leaving the railway domain and the real-time deployment characterization largely unexplored. Future work targets the extension of the latency and energy characterization to additional state-of-the-art anomaly segmentation methods, the evaluation of the pipeline on a broader set of embedded platforms, and the exploration of multimodal anomaly segmentation through camera-LiDAR fusion using the calibrated point clouds provided by OSDaR-AR.

References

1. Aromolo, F., Stevanato, A., Biondi, A., Buttazzo, G.: Real-time multitasking of deep neural networks with Nvidia TensorRT. In: 2025 IEEE Real-Time Systems

Symposium (RTSS). pp. 364–377 (2025)
2. Blum, H., Sarlin, P.E., Nieto, J., Siegwart, R., Cadena, C.: The Fishyscapes benchmark: Measuring blind spots in semantic segmentation. International Journal of Computer Vision **129**(11), 3119–3135 (2021)
3. Bogdoll, D., Nitsche, M., Zöllner, J.M.: Anomaly detection in autonomous driving: A survey. In: IEEE/CVF CVPR. pp. 4488–4499 (2022)
4. Chan, R., Lis, K., Uhlemeyer, S., Blum, H., Honari, S., Siegwart, R., Fua, P., Salzmann, M., Rottmann, M.: SegmentMeIfYouCan: A benchmark for anomaly segmentation. arXiv:2104.14812 (2021)
5. Cordts, M., Omran, M., Ramos, S., Rehfeld, T., et al.: The Cityscapes dataset for semantic urban scene understanding. In: IEEE/CVF CVPR. pp. 3213–3223 (2016)
6. Jung, S., Lee, J., Gwak, D., Choi, S., Choo, J.: Standardized max logits: A simple yet effective approach for identifying unexpected road obstacles in urban-scene segmentation. In: IEEE/CVF ICCV. pp. 15425–15434 (2021)
7. Lin, S.C., Zhang, Y., Hsu, C.H., Skach, M., Haque, M.E., Tang, L., Mars, J.: The architectural implications of autonomous driving: Constraints and acceleration. In: International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 751–766 (2018)
8. Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft COCO: Common objects in context. In: European Conference on Computer Vision (ECCV). pp. 740–755. Springer (2014)
9. Lis, K., Nakka, K., Fua, P., Salzmann, M.: Detecting the unexpected via image resynthesis. In: IEEE/CVF ICCV. pp. 2152–2161 (2019)
10. Liu, W., Wang, X., Owens, J., Li, Y.: Energy-based out-of-distribution detection. Advances in Neural Information Processing Systems **33**, 21464–21475 (2020)
11. Nayal, N., Yavuz, M., Henriques, J.F., Güney, F.: RbA: Segmenting unknown regions rejected by all. In: IEEE/CVF ICCV. pp. 711–722 (2023)
12. Nesti, F., D'Amico, G., Marinoni, M., Buttazzo, G.: OSDaR-AR: Enhancing railway perception datasets via multi-modal augmented reality. arXiv:2602.22920 (2026)
13. Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., et al.: DINOv2: Learning robust visual features without supervision. arXiv:2304.07193 (2023)
14. Pinggera, P., Ramos, S., Gehrig, S., Franke, U., Rother, C., Mester, R.: Lost and found: detecting small road hazards for self-driving vehicles. In: IEEE/RSJ IROS. pp. 1099–1106. IEEE (2016)
15. Shoeb, Y., Nowzad, A., Gottschalk, H.: Out-of-distribution segmentation in autonomous driving: Problems and state of the art. In: IEEE/CVF CVPR. pp. 4310–4320 (2025)
16. Siméoni, O., Vo, H.V., Seitzer, M., Baldassarre, F., Oquab, M., Jose, C., Khalidov, V., Szafraniec, M., Yi, S., et al.: DINOv3. arXiv:2508.10104 (2025)
17. Turkcan, M.K.: Detect anything in real time: From single-prompt segmentation to multi-class detection. arXiv:2603.11441 (2026)
18. Vojíř, T., Jan, Š., Matas, J.: PixOOD: Pixel-level out-of-distribution detection. IEEE Transactions on Pattern Analysis and Machine Intelligence (2026)
19. Zendel, O., Murschitz, M., Zeilinger, M., Steininger, D., Abbasi, S., Beleznai, C.: RailSem19: A dataset for semantic rail scene understanding. In: IEEE/CVF CVPR Workshops. pp. 1221–1229 (2019)
20. Zhao, W., Li, J., Dong, X., Xiang, Y., Guo, Y.: Segment every out-of-distribution object. In: IEEE/CVF CVPR. pp. 3910–3920 (2024)